

20 Newsgroups Text Classification with Word2Vec, TF-IDF, BERT, MLM, and Llama-3 + QLoRA

*This work is submitted as part of the final project submission for the NALAPRO course.

Nilabh Pandey

School of Computer Science and Information Technology

Lucerne University of Applied Sciences and Arts

Lucerne, Switzerland

nilabh.pandey@stud.hslu.ch

Abstract—Sorting text into the right topic is one of the oldest problems in natural language processing, and it is still not easy. In this project, I take a single, well-known dataset, the 20 Newsgroups corpus, and I try to solve the same task with four different families of methods. I start simple, with word features such as Word2Vec and TF-IDF passed to a small neural network. I then move to a fine-tuned BERT model, and after that to a version of BERT that is first adapted to the domain with masked language modeling. Finally, I use a large language model, Llama-3, both by prompting it without any training and by fine-tuning it efficiently with QLoRA. My goal is not to find one winner, but to understand how much each step actually helps on the same problem. The fine-tuned Llama-3 with QLoRA gave the strongest result at about 78% accuracy, followed by the BERT models at around 71%, while the classical features reached about 70%. Prompting the large model without training was far weaker at about 51%. Along the way I found two things worth sharing: domain adaptation gave BERT a small but real boost, and for the large language model, the format of the prompt mattered far more than the number of examples I showed it. All of my runs were logged to Weights and Biases so that the results can be checked and repeated.

Index Terms—text classification, 20 Newsgroups, Word2Vec, TF-IDF, BERT, masked language modeling, Llama-3, QLoRA

I. INTRODUCTION

Imagine a large pile of old forum posts, and the job is to put each one under the right topic, like hockey, space, or religion. A human can do this quickly for a few posts, but not for tens of thousands. This is the kind of task that text classification is built for, and the 20 Newsgroups dataset has been a standard test bed for it for many years.

The dataset is harder than it looks. There are twenty topics, and several of them are close to each other. Three of them are about religion, three are about politics, and a whole group is about computers. A post about a religious argument can easily look like a post about politics. Because there are twenty classes and they are roughly the same size, a model that guesses at random would only be right about five percent of the time. Any real accuracy has to be learned from the words in the posts.

The way people solve this kind of problem has changed a lot. In the early days, the common approach was to turn

each document into a vector of word counts or weights, such as TF-IDF, and feed that to a simple classifier. Later, learned word vectors like Word2Vec added a sense of meaning, so that words used in similar ways ended up close together. Then came pre-trained transformer models such as BERT, which read a sentence in both directions and can be fine-tuned for a task. Most recently, some large language models such as Llama-3 can often do a task straight away, just by being asked in plain language, with little or no training.

Instead of picking one of these methods, I decided to try them all on the same dataset, under the same rules, and compare them fairly. The point of the project is to see how much each new idea really adds, and to explain the results rather than just report them. My main contributions are as follows: First, I give a clear, side-by-side comparison of classical features, fine-tuned BERT, domain-adapted BERT, and Llama-3 prompting on 20 Newsgroups. Second, I add an optional QLoRA experiment that fine-tunes Llama-3 cheaply and turns out to be my best model. Third, I share several practical lessons, including the value of domain adaptation, the strong effect of prompt format, and the gap that still exists between prompting and fine-tuning.

II. TOOLS AND REPRODUCIBILITY

Every experiment in this project, with its settings, training curves, and final metrics, was logged to Weights and Biases so that the results can be checked and repeated:

<https://wandb.ai/nilabhpandey7-hslu/nalapro-project>

The code and documentation are kept in the GitHub repository linked in the Code and Data Availability section. The main tools used were Python and PyTorch for all models, the Hugging Face Transformers and PEFT libraries for BERT and Llama-3, the `bitsandbytes` library for 4-bit quantization, `scikit-learn` for the dataset and the evaluation metrics, `Gensim` for training Word2Vec, and `Matplotlib` and `Seaborn` for the plots. Experiments were tracked with Weights and Biases, and the heavier models were trained on Google Colab using T4 and A100 GPUs. Generative AI tools were used as an assistant for

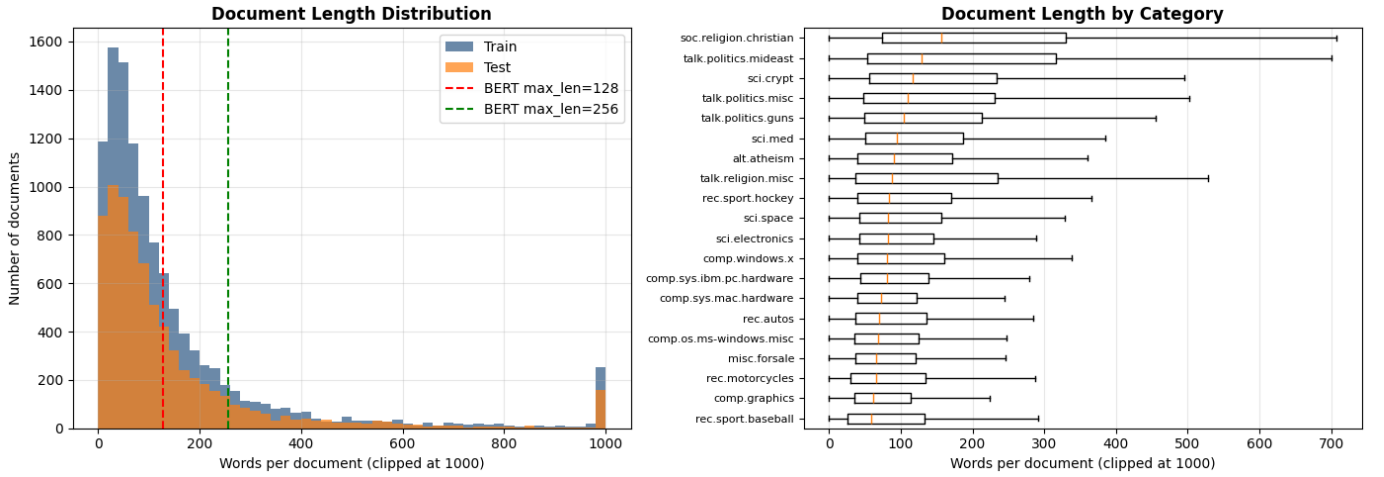


Fig. 1. Document length. Left: the length distribution is strongly right-skewed, with the BERT cut-offs of 128 and 256 tokens marked. Right: length spread by class. Most posts are short, but a few are very long.

coding and writing; this is described in detail in the Generative AI Declaration.

III. BACKGROUND AND RELATED WORK

TF-IDF is a simple but strong baseline, and it is a standard frequency-based representation in the course material [11], [12]. It gives a high weight to a word that appears often in one document but rarely across the whole collection, which makes topic words stand out. Word2Vec [2] takes a different view. It learns a dense vector for each word by looking at the words around it, so that words used in similar contexts end up near each other. A document can then be turned into one vector by averaging the vectors of its words.

BERT [3] is a transformer that was pre-trained on a large amount of text using masked language modeling, where some words are hidden and the model learns to guess them [14]. To use it on a new task, I fine-tune it, which means I keep training all of its weights on my labelled data. A related idea is to keep pre-training the model on text from the same domain before fine-tuning, which often gives a small gain [4].

Large language models such as Llama-3 [6] can be used without any training. I simply describe the task and the model answers. I can also give it a few examples inside the prompt, which is called few-shot learning [5]. The skip-gram and CBOW ideas behind Word2Vec, and the few-shot setup, are also covered in the course notes [13], [15]. When I do want to train such a model, full fine-tuning is expensive because the model is huge. LoRA [7] avoids this by freezing the model and training only small extra matrices. QLoRA [8] goes further and stores the frozen model in 4-bit form, so that an eight billion parameter model can be fine-tuned on a single GPU.

IV. DATASET, PREPROCESSING, AND EXPLORATORY ANALYSIS

I use the 20 Newsgroups corpus as provided by scikit-learn [1]. It holds 18,846 documents spread over 20 topics. Scikit-learn supplies 11,314 of these for training and 7,532

for testing. I keep the test set untouched and split the training portion into 9,051 documents for training and 2,263 for validation, keeping the class balance in each part. For the domain adaptation step in Task 3, I also use a larger set of 17,886 documents without labels, just for the masked language modeling stage.

Before building any model, I looked at the data itself, because the shape of the dataset explains a lot of what happens later.

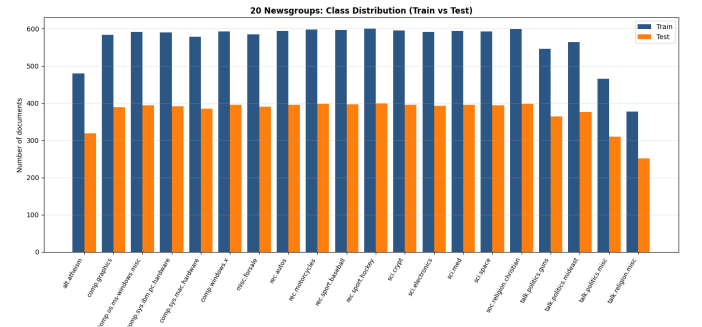


Fig. 2. Number of documents per class in the training and test portions. The classes are close to balanced, so no class weighting is needed.

Figure 2 shows how many posts each topic has. The classes are close to balanced. Most topics have between about 480 and 600 training documents, with `talk.religion.misc` and `alt.atheism` the smallest. Because the classes are so even, I do not need to correct for imbalance, and a model that guessed at random would be right only about five percent of the time. Any real accuracy has to be learned from the words in the posts.

The posts vary a lot in length, as Figure 1 shows. The mean length is about 186 words, but the median is only 83. This large gap means most posts are short while a few are extremely long, the longest being over 11,000 words. This matters for BERT, which can only read a fixed number of tokens. When

the posts are tokenized, about 34% are longer than 128 tokens, but only about 14% are longer than 256. This is why I set the BERT maximum length to 256: it keeps almost all of each post, while a length of 128 would cut off a third of them.

Before training, I remove the headers, footers, and quoted replies from every post. This step matters more than it might seem. These parts of a post often contain the sender name, the email address, or the name of the newsgroup, and a model can use them to guess the label without reading the actual text. By removing them, I force every model to learn from the real content.

The cleaning after that depends on the method, and I made a deliberate choice here. For the classical models in Task 1, I clean the text hard: I lowercase it, and I remove stopwords, numbers, web addresses, and email addresses, following the standard NLP pipeline [9]. A small network has no real knowledge of language, so giving it cleaner input helps. For BERT and Llama-3 I do the opposite. I leave the text close to its natural form and let the model’s own subword tokenizer handle it [10], because these models were pre-trained on normal text and cleaning would throw away signal they know how to use.

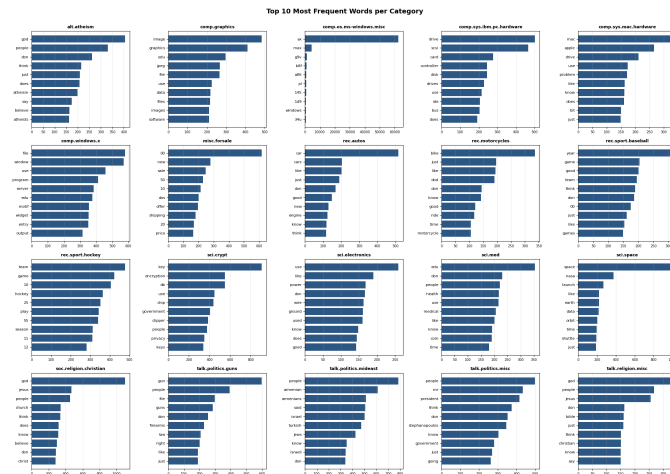


Fig. 3. Word frequency against rank on log-log axes. The near-straight line is the classic shape of Zipf’s law for natural language.

The vocabulary is large but very uneven. The training set has about 1.6 million word tokens but only 46,733 unique words, and almost half of those, 42.2%, appear only once. Figure 3 plots word frequency against rank on log-log axes and gives an almost straight line, which is the classic shape of Zipf’s law for language [12]. The long tail of rare words is the reason I cap the TF-IDF vocabulary with a minimum document frequency, so the network is not fed thousands of words it sees only once.

Figure 4 shows the ten most frequent words in each class once stopwords are removed. Many classes have a clear signature: god and atheism for the religion groups, key and encryption for sci.crypt, car for rec.autos, gun for talk.politics.guns, and space and nasa for sci.space. These strong keywords are exactly what TF-IDF

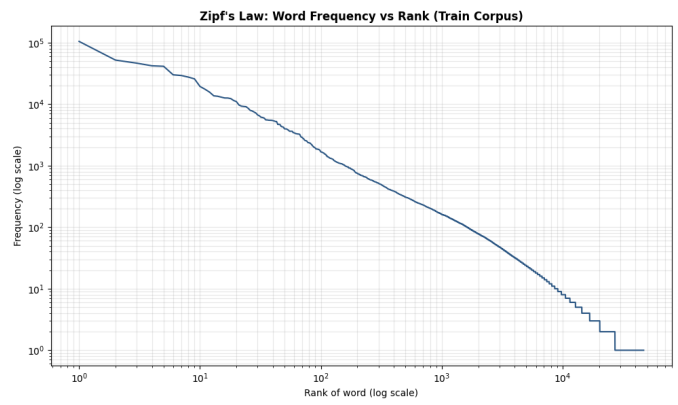


Fig. 4. The ten most frequent words in each class after stopwords removal. Many classes have a clear signature; a few are shaped by quirks of the data.

picks up on, and they explain why even a simple model can separate the clear topics. The plot also reveals a data quirk: comp.os.ms-windows.misc is dominated by the token ax, which comes from a single encoded binary post rather than from real discussion. This is a reminder that the dataset is not perfectly clean.

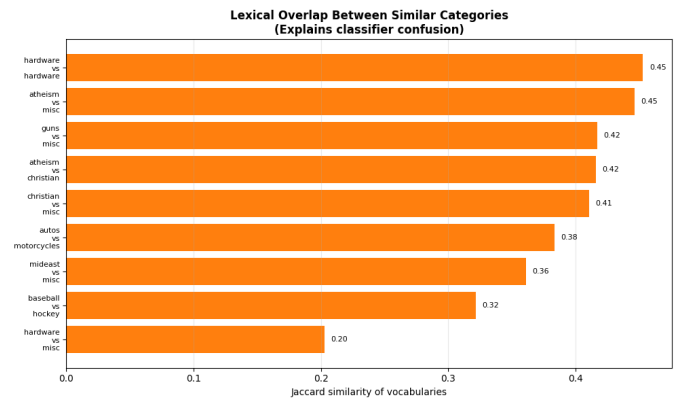


Fig. 5. Jaccard overlap between the vocabularies of similar class pairs. High overlap predicts which classes the models will confuse.

Finally, Figure 5 measures how much vocabulary two similar classes share, using the Jaccard overlap of their word sets. The overlap is high in exactly the places where I would expect trouble. The two hardware classes overlap by 0.45, alt.atheism and talk.religion.misc by 0.45, talk.politics.guns and talk.politics.misc by 0.42, alt.atheism and soc.religion.christian by 0.42, and rec.autos and rec.motorcycles by 0.38. This is the clearest early warning of what every confusion matrix later confirms: the religion classes, the politics classes, and the computing cluster are hard to tell apart because they use the same words. The main weakness of the models is already visible in the data, before any training begins.

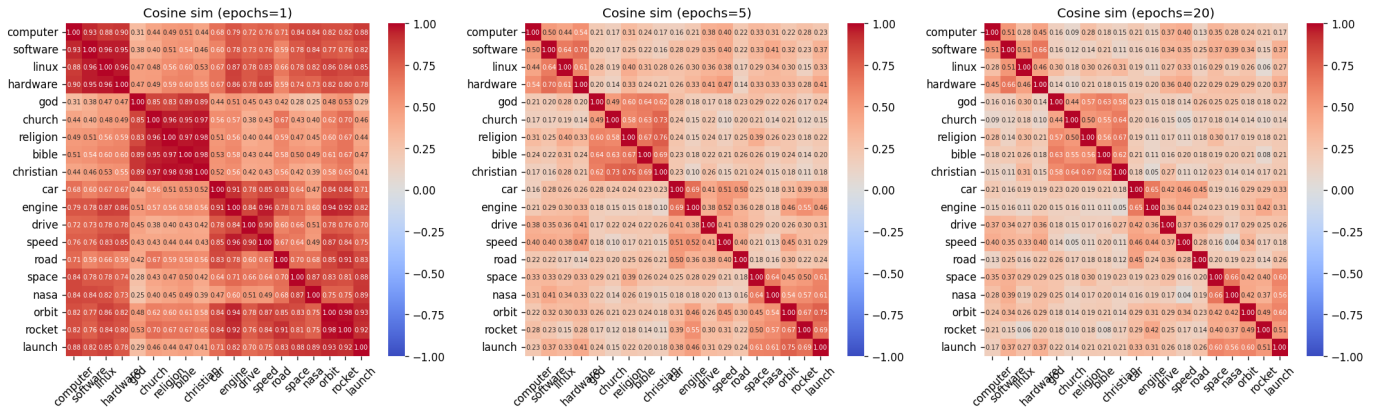


Fig. 6. Cosine similarity between selected words after 1, 5, and 20 training epochs of Word2Vec. At epoch 1 almost everything is similar to everything (wide red region). With more training the space becomes sharper, and only words that share a topic keep a high similarity.

V. TASK 1: CLASSICAL FEATURES AND A SMALL NETWORK

The network in this task is kept very simple on purpose. It has two linear layers with a ReLU activation between them, and a softmax over the twenty classes at the end. I use a hidden size of 256, dropout for regularization, a learning rate of 10^{-3} , and I train for 30 epochs. Because the network never changes, any difference in accuracy comes from the input representation, not the classifier. That is the whole idea of this task.

do the vectors really change as I train for more epochs, and what is the effect? To answer this I looked at the embedding space after 1, 5, and 20 epochs.

Figure 6 shows the clearest answer. It is a set of cosine similarity maps for a group of common words after 1, 5, and 20 epochs. After only one epoch almost every pair of words has a high similarity, shown by the wide red area. The space is crowded and undecided, and even unrelated words look similar. After five and then twenty epochs the picture changes a lot. Most pairs become much less similar, while words that truly belong together stay close. For example the computing words (computer, software, linux, hardware) keep a high mutual similarity, the religion words (god, church, religion, bible, christian) form their own block, and the space words (space, nasa, orbit, rocket, launch) group together. So training does not just make numbers move; it slowly turns a vague space into one with clear topic neighbourhoods.

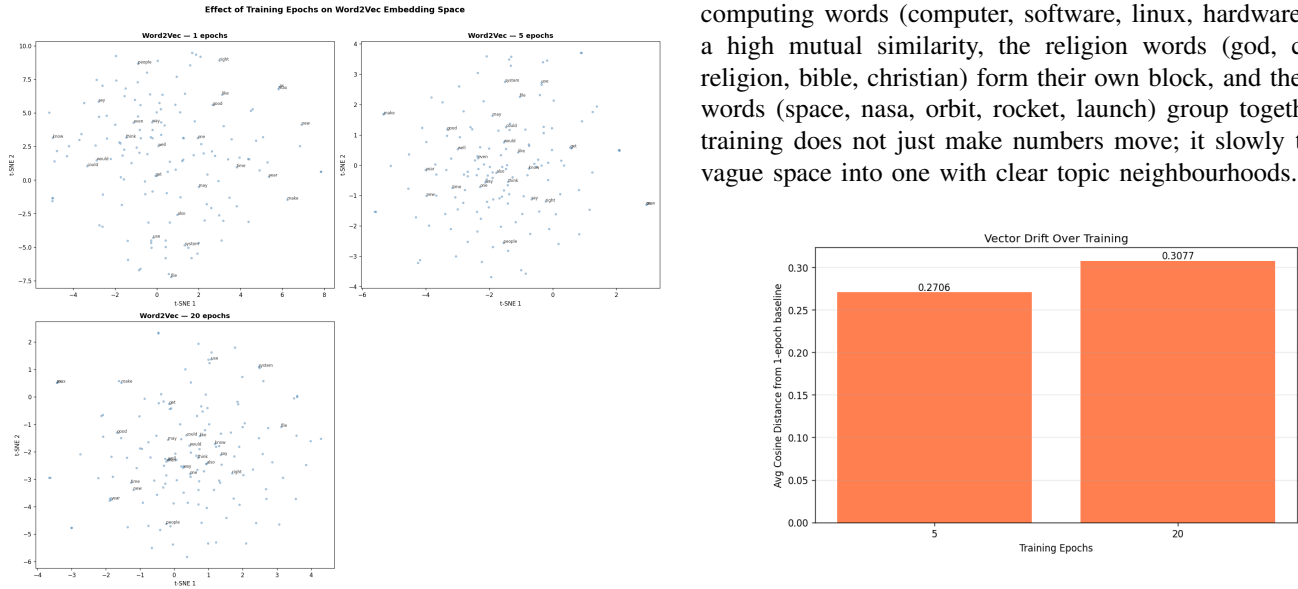


Fig. 7. 2-D projection of the Word2Vec space after 1, 5, and 20 epochs. The arrangement of the same common words changes with training, which shows that the embedding space is reorganising itself.

Fig. 8. Average cosine distance of the word vectors from their one-epoch positions. The space keeps changing as I train longer, but the rate of change slows down.

A. Word2Vec embeddings and how they change with training

For part (b) I train a Word2Vec model on the corpus and represent each document by the average of its word vectors [13], [17]. The assignment asks an interesting question here:

Figure 7 tells the same story from another angle, using a 2-D projection of the vectors. The layout of the common words is clearly different at 1, 5, and 20 epochs, which means the points keep moving as training goes on. To put a number on this, Figure 8 measures how far the vectors have moved away from their one-epoch positions. The average cosine distance

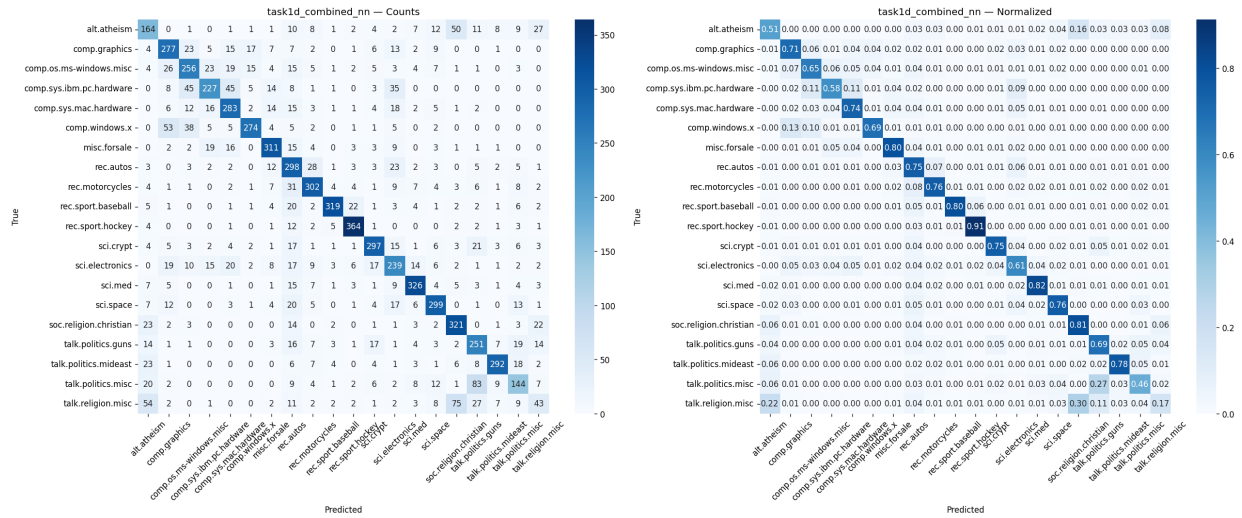


Fig. 9. Confusion matrix for the combined Word2Vec and TF-IDF model (counts on the left, row-normalised on the right). The diagonal is strong for clear topics, but the religion and politics groups bleed into one another.

from the one-epoch baseline grows from about 0.27 at five epochs to about 0.31 at twenty epochs. The vectors are still drifting after five epochs, but the change slows down, which fits what I saw in the similarity maps.

With these embeddings, the network reaches 67.2% accuracy on the test set, with a macro F1 of 0.659. The training curves in Figure 10 look healthy. Training and validation accuracy rise together and then flatten out near 0.71, with only a small gap at the end. This tells me the averaged Word2Vec features are useful but limited, which makes sense, because averaging all the word vectors in a post blurs the strong single keywords that often decide a topic.

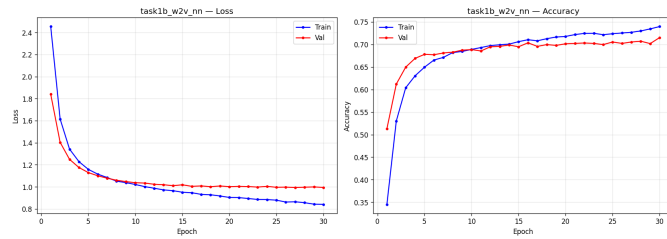


Fig. 10. Training and validation curves for the Word2Vec model. The two lines stay close, which means little overfitting, and accuracy settles near 0.71.

B. TF-IDF features and a comparison

For part (c) I replace the embeddings with TF-IDF vectors and keep the same network. Accuracy goes up to 68.9%, with a macro F1 of 0.678. This beats the averaged Word2Vec features, and the reason is the one I just mentioned. Newsgroup topics are often given away by a few strong words, like "nasa" or "encryption", and TF-IDF keeps those words sharp instead of averaging them away.

But the TF-IDF curves in Figure 11 show a clear problem. The training accuracy climbs close to 0.97, while the validation accuracy peaks early, around epoch five, and then

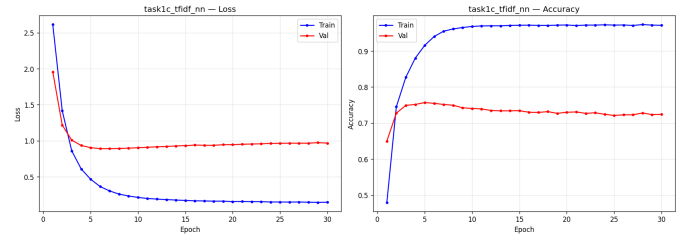


Fig. 11. Training and validation curves for the TF-IDF model. Training accuracy keeps rising while validation peaks early and then falls, which is a clear sign of overfitting.

slowly drops. This is classic overfitting. The TF-IDF input has thousands of sparse features, and the network starts to memorise the training set. The model is strong, but it would do better with early stopping or more regularization. This is a useful lesson and it leads directly into the next part.

C. An experiment to improve the result

For part (d) I needed one more experiment that could improve the result while keeping the same network. I chose to combine the two representations: I join the dense Word2Vec vector and the sparse TF-IDF vector into one input. The idea is that the two signals are different and may help each other. TF-IDF brings the strong keywords, while Word2Vec brings a sense of meaning for posts that use different but related words.

The combined model reaches 70.2% accuracy, with a macro F1 of 0.687, which is the best of the three classical setups. The confusion matrix in Figure 9 shows where the accuracy comes from and where it does not. Clear topics like hockey, baseball, and for-sale posts are handled well. The weak spots are the overlapping groups. The class `talk.politics.misc` is often mixed up with the other politics classes, and `talk.religion.misc` is spread across atheism and Chris-

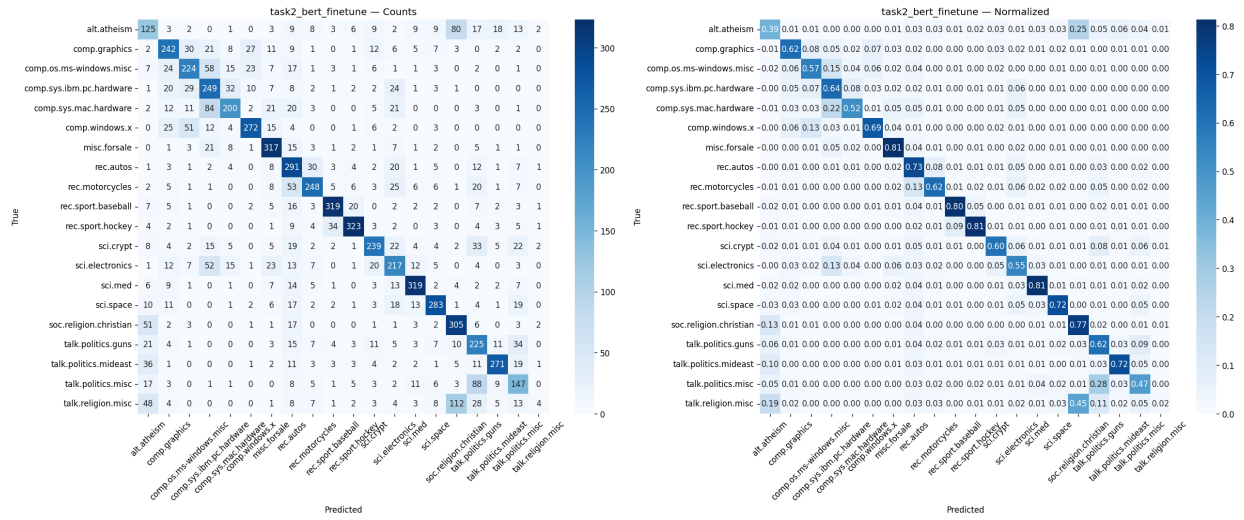


Fig. 12. Confusion matrix for fine-tuned BERT. The model is accurate on most topics. As before, the religion and politics classes are the main source of errors.

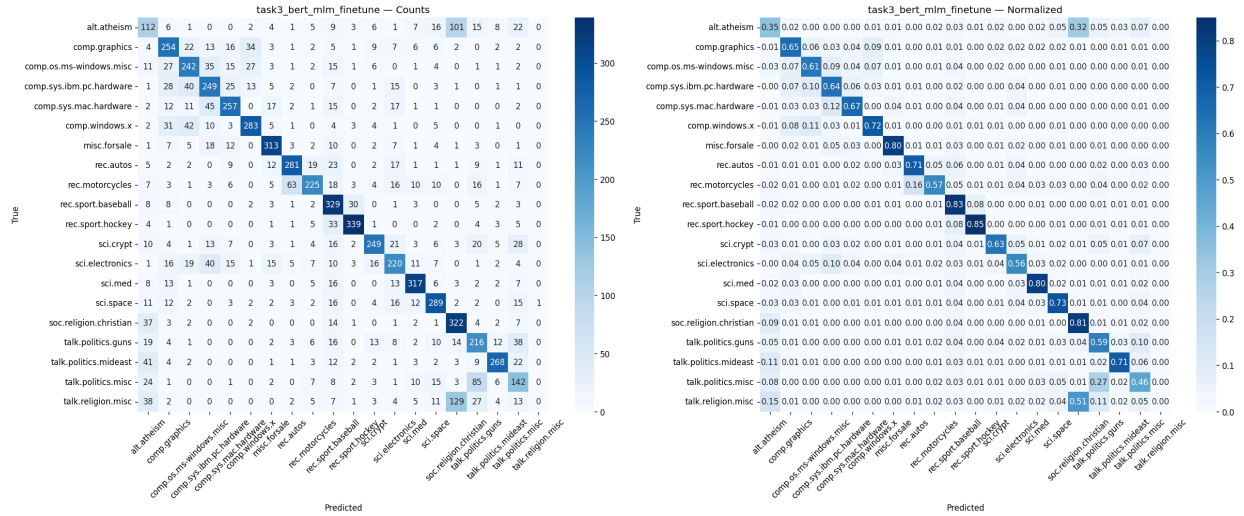


Fig. 13. Confusion matrix for BERT after masked language model domain adaptation and then fine-tuning. The pattern is close to plain BERT, with a small improvement overall.

tianity. These are the same overlaps I noticed in the data, and they will follow me through every later model.

VI. TASK 2: FINE-TUNING BERT

In Task 2, I move from hand-made features to a pre-trained model. I fine-tune `bert-base-uncased` on the training set, which means all of its weights are updated for my task [15]. I use a maximum sequence length of 256 tokens, a batch size suited to the GPU, four epochs, a learning rate of 2×10^{-5} , and the AdamW optimizer. The validation set is used to watch the training.

BERT reaches 71.1% accuracy with a macro F1 of 0.698, which is the best result so far. It passes every classical model. The reason is that BERT reads words in context and in both directions, so it understands that the same word can mean different things in different posts, something a bag of words

or an average vector cannot do. The confusion matrix in Figure 12 shows a strong diagonal, with the usual trouble in the religion and politics corner.

I also learned something about compute. An early version of this model, trained on a laptop CPU with a short sequence length and only one epoch, reached only 64.2%. The same model trained properly on a GPU, with a longer sequence length and more epochs, reached 71.1%. That is a gain of about seven points, and it did not come from a better model. It came from giving the same model enough sequence length and enough training. This is a reminder that compute and basic settings can matter as much as the choice of model.

VII. TASK 3: DOMAIN ADAPTATION WITH MASKED LANGUAGE MODELING

Task 3 asks whether I can do better by first teaching BERT the language of this dataset. I do this in two steps. First I keep pre-training BERT on the newsgroup text using masked language modeling, where 15% of the tokens are hidden and the model learns to fill them in. This uses only the text, not the labels. Then I take this adapted model and fine-tune it on the classification task, just like in Task 2.

The first step works. The masked language modeling perplexity drops from about 15.9 to about 11.2, which means the model became less surprised by newsgroup text [16] and learned its style and vocabulary. After fine-tuning, the model reaches 71.5% accuracy, with a macro F1 of 0.694. This is my best supervised model and a small step above plain BERT. The gain is modest, which is honest and expected for a dataset of this size, but it is real and it agrees with earlier work on domain adaptation [4]. The confusion matrix in Figure 13 looks very much like the BERT one, with slightly cleaner behaviour on a few of the science and sports classes.

VIII. TASK 4: LLAMA-3 ZERO-SHOT AND FEW-SHOT

In Task 4, I try a very different approach. Instead of training a model, I ask a large language model, Llama-3 with eight billion parameters, to do the task directly. I load it in 4-bit form so it fits on one GPU. I give it a system message that lists the twenty categories and asks it to reply with exactly one of them. In the zero-shot setting I just give it the post. In the few-shot setting I add three example posts with their correct labels before the real one. To keep the cost reasonable, I test all of the Llama-3 methods on a fixed sample of 200 test posts, so that every version is judged on the same posts.

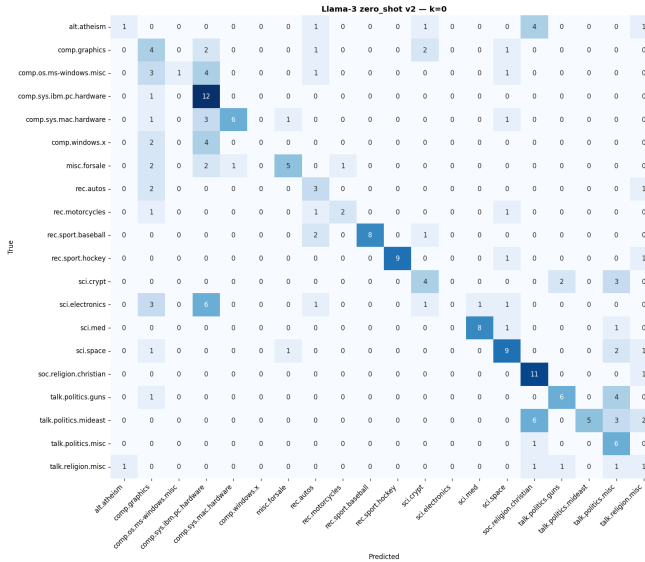


Fig. 14. Confusion matrix for Llama-3 zero-shot, after I fixed the prompt format. The model handles clear topics well and struggles with the overlapping ones.

Zero-shot prompting reaches 51.5% accuracy with a macro F1 of 0.475. This is far below BERT, but it is also remarkable, because the model was never trained on this task and still does ten times better than random across twenty classes. The confusion matrix in Figure 14 shows that the model is reasonable on clear topics but tends to fall back on a few broad categories when it is unsure.

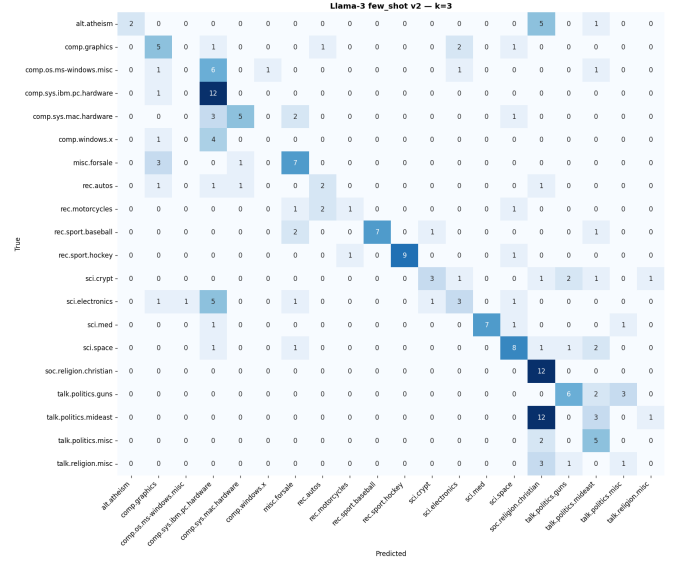


Fig. 15. Confusion matrix for Llama-3 few-shot with three examples. Adding examples did not help, and the model leans toward the few categories it was shown.

The surprising part is that few-shot did worse, not better, at 47.5% accuracy and a macro F1 of 0.420. Adding examples hurt. There are two reasons. The first and biggest reason was the prompt format. My first version sent the model a plain text prompt, and about 22.5% of its answers could not be matched to any category, because the model rambled or repeated the prompt. When I switched to Llama-3's own chat template, which wraps the system and user messages in the special format the model was trained on, the unreadable answers dropped to about one percent. That single change explained most of the improvement, far more than any example did. The second reason is that three random examples cannot represent twenty classes. They only show the model a few topics, which pulls its answers toward those topics and adds noise. Since the instruction was already clear, the examples did more harm than good. Figure 15 shows the few-shot result.

IX. BONUS: FINE-TUNING LLAMA-3 WITH QLoRA

Prompting showed that Llama-3 has useful knowledge but does not use it well on its own. So for the optional bonus I asked a simple question: what happens if I actually train it, but cheaply? I used QLoRA. The model stays frozen in 4-bit form, and I add small low-rank adapter matrices, with rank 16 and alpha 32, to the attention and feed-forward layers. Only about 0.2% of the weights are trained. I turned the task into an instruction, where the model learns to output the category

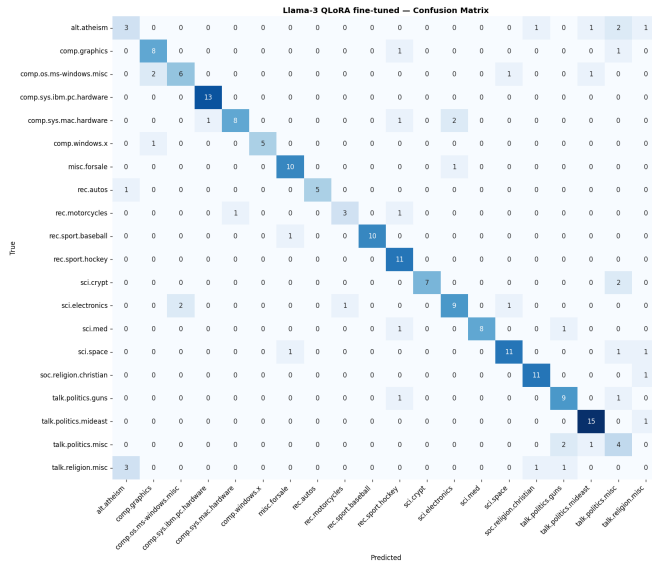


Fig. 16. Confusion matrix for Llama-3 fine-tuned with QLoRA on the 200-post test sample. The diagonal is much stronger than in the prompting results, and most topics are now handled well.

name, and I computed the loss only on the answer tokens, so the model learns to produce the label and not to repeat the prompt. I trained with a learning rate of 2×10^{-4} and the paged 8-bit AdamW optimizer.

The result was the best in the whole project. QLoRA reached about 78.0% accuracy on the 200-post test sample, clearly above both prompting and the BERT models. The confusion matrix in Figure 16 shows a strong, clean diagonal. Topics that the prompted model failed on, such as hardware, the Middle East politics class, and Christianity, are now mostly correct. The model still struggles with the two hardest “misc” classes, `talk.politics.misc` and `talk.religion.misc`, which makes sense, because those classes are vague even for a person. So training only a tiny part of the large model was enough to turn a weak prompted system into the strongest model I built.

I should be fair about one point. The Llama-3 results, including QLoRA, were measured on a 200-post sample, while the classical and BERT models were measured on the full test set of 7,532 posts. So the 78% and the 71.5% are not measured on exactly the same posts. The QLoRA number is still a strong signal, but a fully fair comparison would test QLoRA on the whole test set, which I list as future work.

X. OVERALL COMPARISON AND DISCUSSION

Table I brings all of the results together. Reading it from top to bottom tells the story of the project.

A few clear lessons come out of this work. The first is that combining features helps. The dense Word2Vec vector and the sparse TF-IDF vector each miss something, and together they did better than either alone. The second is that context beats counting. BERT, which reads words in context, passed every classical model, even though the classical models had access

to the same data. The third is that domain adaptation gives a small but honest gain. Teaching BERT the language of the dataset first, with masked language modeling, lifted it slightly and lowered its perplexity, which shows the adaptation was real.

The fourth lesson is about the large language model, and it is the most interesting one. On its own, by prompting, Llama-3 was weak. It even got worse when I added examples, because the format of the prompt mattered much more than the examples did. Fixing the prompt to use the model’s own chat template was the single most useful change in all of Task 4. But once I actually trained the model, even by changing only a tiny fraction of its weights with QLoRA, it became the strongest method in the project. So the headline is not that the biggest model always wins. The headline is that a big model needs the right method around it. Prompted, it trailed a small network. Fine-tuned, it led the field.

TABLE I
TEST RESULTS FOR EVERY METHOD. THE LLAMA-3 METHODS WERE MEASURED ON A FIXED 200-POST SAMPLE; THE OTHERS ON THE FULL TEST SET.

Method	Accuracy	Macro-F1	Train data
Word2Vec + NN	67.2%	0.659	9,051
TF-IDF + NN	68.9%	0.678	9,051
Combined + NN	70.2%	0.687	9,051
BERT fine-tune	71.1%	0.698	9,051
BERT + MLM	71.5%	0.694	9,051 + 17,886
Llama-3 zero-shot	51.5%	0.475	0
Llama-3 few-shot ($k=3$)	47.5%	0.420	3 examples
Llama-3 QLoRA	78.0%	0.746	~3,000

Finally, almost every model failed in the same place. The religion classes and the politics classes overlap in their words, and so they overlap in the predictions. This was visible in the data before I trained anything, and it stayed visible in every confusion matrix. It is a property of the dataset, not a bug in one model.

XI. LIMITATIONS AND FUTURE WORK

There are a few clear ways this work could be improved. The most important one is the evaluation gap I already mentioned: the Llama-3 methods were scored on a 200-post sample, while the other models were scored on the full test set. Testing QLoRA on the whole test set would make the comparison exact.

The TF-IDF and combined models overfit, so adding early stopping or stronger regularization would likely raise their validation accuracy. The few-shot examples were picked at random, and choosing examples that are similar to each new post would probably make few-shot helpful instead of harmful. For BERT, trying a stronger encoder such as RoBERTa or training the masked language model stage for longer could push the supervised result higher. And since the religion and politics classes are the main weakness across the board, a method that focuses on telling those few classes apart would

help every model at once. Scaling the QLoRA training to more data and more epochs is another natural next step.

XII. CONCLUSION

I set out to solve one text classification task with four different families of methods and to understand the results, not just collect them. Accuracy rose as I added task-specific training, from about 67% for simple word features, to about 71% for fine-tuned and domain-adapted BERT. A large language model on its own, through prompting, reached only about 51%, but the same model fine-tuned cheaply with QLoRA reached about 78% and became my best system. The clearest message is that the method has to match the data and the compute. A small model with good features is a solid baseline, a fine-tuned BERT is a strong and reliable choice, and a large language model is only as good as the way I adapt it. I also learned, in a very practical way, that domain adaptation gives a real if small gain, and that for a large model the shape of the prompt can matter more than the examples inside it.

XIII. CODE AND DATA AVAILABILITY

All of the code, the documentation, and this report are available in a public GitHub repository, which has been shared with the lecturer for evaluation:

<https://github.com/ProNilabh/nalapro-project>

The final report is located in the repository at `docs/report.pdf`. The dataset is not included in the repository, as required; it is downloaded directly through scikit-learn's `fetch_20newsgroups` loader. Every experiment, with its settings, training curves, and final metrics, was logged to Weights and Biases and can be inspected at <https://wandb.ai/nilabhpandey7-hslu/nalapro-project>.

XIV. GENERATIVE AI DECLARATION

In line with the course policy, I declare that generative AI tools were used during this project. I used them as an assistant for writing and debugging code, for explaining unfamiliar parts of libraries such as Hugging Face, Transformers and PEFT, and for improving the clarity and wording of this report. I did not use AI to generate my results. The design of the experiments, the choice of models and settings, the training runs, and the analysis of the outputs is my own work. Every result reported here was produced by my own code and logged to Weights and Biases. I have reviewed and understood all of the code and the content in this report, and I can explain and defend each part of it.

REFERENCES

- [1] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [2] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," in *Proc. ICLR Workshop*, 2013.
- [3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [4] S. Gururangan et al., "Don't stop pretraining: Adapt language models to domains and tasks," in *Proc. ACL*, 2020, pp. 8342–8360.
- [5] T. Brown et al., "Language models are few-shot learners," in *Proc. NeurIPS*, 2020, pp. 1877–1901.
- [6] A. Grattafiori et al. (Meta AI), "The Llama 3 herd of models," arXiv:2407.21783, 2024.
- [7] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022.
- [8] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Proc. NeurIPS*, 2023.
- [9] F. Shahbazi, "NLP Pipeline," Lecture notes for NALAPRO, Dept. of Information Technology, Lucerne Univ. of Applied Sciences and Arts (HSLU), Lucerne, Switzerland.
- [10] F. Shahbazi, "Tokenization," Lecture notes for NALAPRO, Dept. of Information Technology, HSLU, Lucerne, Switzerland, 2026.
- [11] F. Shahbazi, "Explicit Semantic Analysis," Lecture notes for NALAPRO, Dept. of Information Technology, HSLU, Lucerne, Switzerland.
- [12] F. Shahbazi, "N-Gram Detection (Frequency-based)," Lecture notes for NALAPRO, Dept. of Information Technology, HSLU, Lucerne, Switzerland.
- [13] F. Shahbazi, "N-Gram Detection (Prediction-based) — Word2Vec," Lecture notes for NALAPRO, Dept. of Information Technology, HSLU, Lucerne, Switzerland.
- [14] F. Shahbazi, "Transformer, Pretraining, BERT," Lecture notes for NALAPRO, Dept. of Information Technology, HSLU, Lucerne, Switzerland, 2026.
- [15] F. Shahbazi, "Fine-tuning Models: Classification," Lecture notes for NALAPRO, Dept. of Information Technology, HSLU, Lucerne, Switzerland, 2026.
- [16] F. Shahbazi, "Text Generation," Lecture notes for NALAPRO, Dept. of Information Technology, HSLU, Lucerne, Switzerland, 2026.
- [17] F. Shahbazi, "Creating Text Embedding Models," Lecture notes for NALAPRO, Dept. of Information Technology, HSLU, Lucerne, Switzerland, 2026.